

Aula 2 — React Native em Profundidade

"RN era lento. Ainda é em 2026?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 27/05/2026

Abertura — Fluxo Fork + PR (~15min)

Antes do conteúdo técnico, **abrimos o deck de entrega via GitHub:**

 `compartilhado/slides-source/intro-fluxo-pr-github.pdf`

 Repo público: `slides/intro-fluxo-pr-github.pdf`

Cobre: fork → clone → branch → commit → push → PR → link Canvas.

Termina com **demo ao vivo** num repo dummy.

| Padrão de toda Aula 2: garantir que ninguém trava na entrega da próxima atividade.

Recap da Aula 1

- 4 tipos de apps móveis com trade-offs distintos
- Estudos de caso: Airbnb, Discord, Coinbase, Alibaba
- ADRs como artefato de decisão arquitetural
- Setup ambiente RN funcionando

Atividade 1 — ADR Arquitetural entregue? (espero que sim)

Agenda da aula (3h30)

1. **Bloco 1 (75min)** — Old Architecture vs New Architecture
2. **Intervalo (30min)**
3. **Bloco 2 (75min)** — Hands-on: app de feed com RTK Query + Reanimated
4. **Wrap-up (15min)** — Atividade 2

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Compreender** Old vs New Architecture do RN (Bridge → JSI/Fabric/TurboModules)
- **Diferenciar** Hermes engine vs JSC e quando cada um faz sentido
- **Aplicar** React Navigation v7 com stack, tabs e deep linking
- **Implementar** RTK Query com cache invalidation e prefetch
- **Desenvolver** animações performáticas com Reanimated v3 worklets

Por que RN ainda gera dúvida?

| "RN era lento em 2018. Mudou?"

Sim, drasticamente. Mas a má fama persiste.

A New Architecture (oficial 2022+) reescreveu a **comunicação JS ↔ nativo** do zero:

-  Bridge assíncrono com serialização JSON (lento, ineficiente)
-  JSI (JavaScript Interface) — chamadas síncronas via referência C++
-  Fabric — novo renderer concorrente
-  TurboModules — native modules com Codegen
-  Hermes — bytecode pré-compilado

Old Architecture — como era

JS Thread

React reconciler



Native Thread

UIKit / Android Views



BRIDGE (assíncrono)
Serializa tudo em JSON e envia

Problema: cada chamada vira mensagem
Bottleneck: scroll com many items
Animação: depende de ida-e-volta

New Architecture — JSI

JSI (JavaScript Interface): ponte de baixo nível em C++.

```
// Native side expõe HostObject
class MyTurboModule : public HostObject {
    jsi::Value get(jsi::Runtime& rt, const PropNameID& name) override {
        // ...
    }
};

// JS side acessa via referência direta
import { MyTurboModule } from 'react-native-my-module';
const result = MyTurboModule.getValue();
// SÍNCRONO. Sem JSON. Sem bridge.
```

Resultado: latência de chamada cai de ms para μ s.

New Architecture — Fabric

Fabric: novo renderer com **commit phase concorrente**.

- Antes: layout + render acoplados em JS thread
- Agora: layout em background, render sincronizado com UI thread
- Suporte a **suspense** e **concurrent mode** do React 18+

Resultado: **menos jank**, especialmente em telas com listas e animações simultâneas.

New Architecture — TurboModules + Codegen

Codegen: ferramenta que **gera bindings** automaticamente.

```
// 1. Definir spec em TS
import { TurboModule, TurboModuleRegistry } from 'react-native';

export interface Spec extends TurboModule {
  getValue(arg: string): Promise<number>;
  getArray(): { items: string[] };
}

export default TurboModuleRegistry.getEnforcing<Spec>('MyModule');
```

```
# 2. Codegen gera Kotlin + Swift bindings
yarn codegen
```

Tipos consistentes entre JS, Android e iOS. Adeus, bugs de `Maybe<string>` virando `null`.

Hermes engine

Hermes é engine JS otimizado para **mobile**, criado pela Meta.

- Bytecode pré-compilado (`.hbc`) → cold start mais rápido
- Memória sob controle em devices low-end
- Garbage collection otimizado para apps RN
- Padrão em RN 0.70+

```
// Verificar Hermes ativo em runtime
console.log('Hermes ativo?', global.HermesInternal !== null);

// Em CLI (build):
// Android: app/build.gradle → enableHermes: true (ou hermesEnabled em RN 0.70+)
// iOS: Podfile → :hermes_enabled => true
```

Antes (JSC): ~2.5s cold start em Android mid-range. Com Hermes: ~1.4s.

React Navigation v7 — stack + tabs

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

const Stack = createNativeStackNavigator();
const Tabs = createBottomTabNavigator();

function RootStack() {
  return (
    <Stack.Navigator>
      <Stack.Screen name="Home" component={HomeTabs} />
      <Stack.Screen name="Detail" component={DetailScreen} />
    </Stack.Navigator>
  );
}
```

`createNativeStackNavigator` usa **navigation nativa** (UINavigationController iOS, Fragment Android), não emula em JS.

React Navigation — Deep linking

```
const linking = {
  prefixes: ['myapp://', 'https://myapp.com'],
  config: {
    screens: {
      Home: '',
      Detail: 'item/:id',
      Profile: 'user/:username',
    },
  },
};

<NavigationContainer linking={linking}>
  {/* ... */}
</NavigationContainer>
```

Deep links funcionam:

- iOS: Universal Links + Custom URL schemes
- Android: App Links + Intent filters

RTK Query — data fetching estruturado

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';

export const moviesApi = createApi({
  reducerPath: 'moviesApi',
  baseQuery: fetchBaseQuery({
    baseUrl: 'https://api.themoviedb.org/3/',
    prepareHeaders: (headers) => {
      headers.set('Authorization', `Bearer ${TOKEN}`);
      return headers;
    },
  }),
  tagTypes: ['Movie'],
  endpoints: (builder) => ({
    getMovies: builder.query<Movie[], void>({
      query: () => 'movie/popular',
      providesTags: ['Movie'],
    }),
    getMovie: builder.query<Movie, number>({
      query: (id) => `movie/${id}`,
    }),
  }),
});

export const { useGetMoviesQuery, useGetMovieQuery } = moviesApi;
```

RTK Query — uso em componente

```
function MovieList() {  
  const { data, isLoading, error, refetch } = useGetMoviesQuery();  
  
  if (isLoading) return <ActivityIndicator />;  
  if (error) return <Text>Error: {JSON.stringify(error)}</Text>;  
  
  return (  
    <FlatList  
      data={data}  
      keyExtractor={(item) => String(item.id)}  
      renderItem={({ item }) => <MovieCard movie={item} />}  
      onRefresh={refetch}  
      refreshing={isLoading}  
    />  
  );  
}
```

Cache, retry, dedup, invalidation — tudo gerenciado.

Reanimated v3 — worklets

Worklet: função JS que roda na UI thread, não na JS thread.

```
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from 'react-native-reanimated';

function AnimatedBox() {
  const offset = useSharedValue(0);

  const style = useAnimatedStyle(() => {
    return {
      transform: [{ translateX: offset.value }],
    };
  });

  return (
    <>
      <Animated.View style={[styles.box, style]} />
      <Button onPress={() => (offset.value = withSpring(100))} title="Move" />
    </>
  );
}
```


Worklet — cuidado com `runOnJS`

```
const handleAnimationEnd = () => {  
  // Função normal — roda em JS thread  
  navigation.navigate('Detail');  
};  
  
const style = useAnimatedStyle(() => {  
  if (offset.value >= 100) {  
    runOnJS(handleAnimationEnd)(); // salta de UI thread → JS thread  
  }  
  return {  
    transform: [{ translateX: offset.value }],  
  };  
});
```

`runOnJS` reintroduz overhead. Use só quando precisar — não em hot path.

Estado: Redux Toolkit vs alternativas

Lib	Quando usar	Tamanho bundle
Redux Toolkit + RTK Query	Apps com data fetching estruturado e cache	~50KB
Zustand	State leve, sem boilerplate Redux	~3KB
Jotai	Atomic state, granular	~7KB
MobX	OOP-like, observable	~50KB
TanStack Query (sem Redux)	Só data fetching, sem global state	~30KB

Não há resposta universal. Trade-off entre features e tamanho.

Demo — IA gerando RTK slice

Prompt para Claude:

```
Crie um slice RTK Query completo para a TMDB API com:  
- baseQuery configurado com authorization header  
- 4 endpoints: getPopularMovies, getMovie, searchMovies, getGenres  
- TypeScript types completos  
- tagTypes: Movie, Genre  
- Cache invalidation correto  
  
Output em um único arquivo.
```

IA é boa em scaffolding tipado. Mas **revise sempre** — alucinações de schema são comuns.

Intervalo — 30min

Volta às :__

Aproveita pra:

- Esticar perna
- Verificar Node 22 + Expo CLI + simulador funcionando
- Tomar café / água

Próximo bloco é hands-on com RTK Query + Reanimated. Já abre terminal + IDE.

Hands-on — vamos fazer juntos (75min)

Roteiro

1. Clone starter `aula-02` (5min)
2. Configurar TMDB API token (5min)
3. Criar slice RTK Query com IA assistida (15min)
4. Tela `MovieList` com FlatList + RTK Query hook (15min)
5. Tela `MovieDetail` via deep link (15min)
6. Animação Reanimated em transição entre telas (20min)
7. Persistência de favoritos com MMKV (10min)
8. Discussão (5min)

Pitfalls comuns

- ✗ `useEffect` para data fetching em vez de RTK Query — bug magnet
- ✗ Animated API legado em vez de Reanimated — jank em apps complexos
- ✗ `AsyncStorage` para tudo — async + frágil; preferir MMKV (síncrono, 30x mais rápido)
- ✗ Esquecer de habilitar New Architecture — deixa de usar JSI/Fabric
- ✗ `useState` global em vez de RTK — re-render desnecessário em árvore inteira

Pitfalls — performance específica

- ✗ **Inline styles em FlatList** — recria object a cada render
- ✗ **Não usar `getItemLayout`** quando altura é constante
- ✗ **Esquecer `keyExtractor`** — fallback usa index, quebra animações
- ✗ **Image sem cache policy** — explode memória; usar `react-native-fast-image` ou `expo-image` com cache + dimensões fixas + `Image.prefetch()` em hot paths
- ✗ **Worklet chamando função JS pesada** — derrota o propósito

Atividade assíncrona — Atividade 2: App RN (15 pts)

Entrega: até 09/06.

Estender app da aula com:

1. **Tela de favoritos** persistente em MMKV
2. **Animação Reanimated não trivial** (gesture-driven ou shared element)
3. **Medição de TTI** via Flipper / React DevTools profiler
4. **Screencast 2min** mostrando golden path e edge cases

Critérios detalhados de avaliação: Canvas (módulo Atividade 2).

Leituras obrigatórias (pré-Aula 3)

- **Eisenman** — *Learning React Native* (O'Reilly), cap 4–7
- **React Native New Architecture RFC** (Meta, GitHub oficial) — leitura técnica
- **Software Mansion** — *Reanimated 3 Internals* (artigo)
- **Meta Engineering Blog** — *Hermes JavaScript Engine*

Próxima aula (10/06)

Aula 3 — Cross-Platform Comparativo + Native Bridging

Vamos construir native module em Kotlin (Android) + Swift (iOS) **em sala**.

Pré-requisito:

- Atividade 2 entregue (09/06)
- Xcode + Android Studio instalados e funcionais
- Ler post-mortem completo da Airbnb (5 partes)

Obrigado!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Próxima quarta, 10/06, 19:00